Playwright JS/TS Automation Testing from Scratch & Framework Notes

TABLE OF CONTENTS

1. Introduction to Playwright
2. Installation & Setup
3. First Playwright Test
4. Locators in Playwright
5. Basic Actions
6. Assertions & Validations
7. Waits & Synchronization
8. Handling Web Elements
9. Frames & iFrames
10. Alerts, Popups & Dialogs
11. Screenshots & Videos
12. Network Interception
13. API Testing with Playwright
14. Page Object Model (POM)
15. Test Configuration
16. Hooks & Fixtures
17. Data-Driven Testing
18. Parallel Execution
19. Reporting
20. Debugging Tests
21. JavaScript Essentials for Playwright
22. **Interview Questions & Answers (NEW)**
23. **Resume Guide for Playwright Testers (NEW)**

Module 1: Introduction to Playwright

What is Playwright?

Playwright is a modern automation framework developed by Microsoft for testing web applications.

RAHUL SHETTY ACADEMY

Key Features:

- Multi-browser support - Chrome, Firefox, Safari (WebKit)
- Multi-language support - JavaScript, TypeScript, Python, Java, .NET
- Auto-wait mechanism - No need for explicit waits
- Fast execution - Parallel test execution
- Mobile emulation - Test on mobile viewports
- Network interception - Mock API responses
- Screenshots & Videos - Built-in screenshot and video recording
- Trace viewer - Debug failed tests easily

Why Playwright over Selenium?

Feature	Playwright	Selenium
Speed	Fast	Slower
Auto-wait	Built-in	Manual
API Testing	Yes	No
Network Interception	Yes	Limited
Browser Contexts	Yes	No
Parallel Execution	Easy	Complex setup

Installation & Setup

Step 1: Initialize Node Project

npm init -y

Step 2: Install Playwright

npm init playwright@latest

This will:

- Install Playwright and browsers
- Create playwright.config.js

RAHUL SHETTY ACADEMY

- Create tests folder
- Create example test files

Step 3: Install Browsers

npx playwright install

Project Structure:

```
project/
├── tests/
│   └── example.spec.js
├── playwright.config.js
├── package.json
└── node_modules/
```

Module 2: First Playwright Test

Basic Test Structure

```
const { test, expect } = require('@playwright/test');

test('My first test', async ({ page }) => {
   // Navigate to URL
   await page.goto('https://rahulshettyacademy.com/loginpagePractise/');

   // Get page title
   console.log(await page.title());

   // Verify title
   await expect(page).toHaveTitle('LoginPage Practise | Rahul Shetty Academy');
});
```

Test Anatomy:

1. **Import** - Import test and expect from Playwright
2. **test()** - Define test case with description
3. **async ({ page })** - Page fixture automatically provided
4. **await** - All Playwright actions are async

RAHUL SHETTY ACADEMY

5. **expect** - Assertion library

Running Tests

```
# Run all tests
npx playwright test

# Run specific test file
npx playwright test tests/login.spec.js

# Run in headed mode (see browser)
npx playwright test --headed

# Run in specific browser
npx playwright test --project=chromium
npx playwright test --project=firefox

# Run in debug mode
npx playwright test --debug

# Run with UI mode
npx playwright test --ui
```

Module 3: Locators in Playwright

Finding Elements

1. CSS Selectors (Most Common)

```
await page.locator('#username').fill('rahulshetty');
await page.locator('.btn-primary').click();
await page.locator('input[name="email"]').fill('test@test.com');
```

2. Text Locators

```
await page.locator('text=Sign In').click();
await page.getByText('Sign In').click();
```

3. Role-based Locators (Recommended)

RAHUL SHETTY ACADEMY



```
await page.getByRole('button', { name: 'Sign In' }).click();
await page.getByRole('textbox', { name: 'Username' }).fill('test');

await page.getByRole('link', { name: 'Home' }).click();
```

4. Placeholder Locators

```
await page.getByPlaceholder('Enter username').fill('test');
```

5. Label Locators

```
await page.getByLabel('Email').fill('test@test.com');
```

6. XPath (Use Sparingly)

```
await page.locator('//input[@id="username"]').fill('test');
```

7. Test ID (Best Practice)

```
// HTML: <button data-testid="submit-btn">Submit</button>
await page.getByTestId('submit-btn').click();
```

Locator Best Practices

Priority Order:

1. getByRole() - Most resilient
2. getByTestId() - For dynamic content
3. getByLabel() - For form fields
4. getByPlaceholder() - For inputs
5. getByText() - For visible text
6. CSS selectors - When above don't work
7. XPath - Last resort

RAHUL SHETTY ACADEMY

Module 4: Basic Actions

Input Actions

Typing Text

```
// Type in input field
await page.locator('#username').fill('rahulshetty');

// Clear and type
await page.locator('#username').clear();
await page.locator('#username').fill('newtext');

// Type one character at a time (with delay)
await page.locator('#search').pressSequentially('Playwright', { delay: 100 });
```

Clicking

```
// Single click
await page.locator('#loginBtn').click();

// Double click
await page.locator('#item').dblclick();

// Right click
await page.locator('#menu').click({ button: 'right' });

// Click with modifier
await page.locator('#link').click({ modifiers: ['Control'] });
```

Checkboxes & Radio Buttons

```
// Check a checkbox
await page.locator('#terms').check();

// Uncheck
await page.locator('#newsletter').uncheck();

// Select radio button
await page.locator('#gender-male').check();

// Verify checked
```

RAHUL SHETTY ACADEMY



```
await expect(page.locator('#terms')).toBeChecked();
```

Dropdowns

```
// Select by value
await page.locator('#country').selectOption('India');

// Select by label
await page.locator('#country').selectOption({ label: 'India' });

// Select by index
await page.locator('#country').selectOption({ index: 2 });

// Select multiple
await page.locator('#countries').selectOption(['India', 'USA']);
```

Navigation Actions

```
// Navigate to URL
await page.goto('https://rahulshettyacademy.com');

// Go back
await page.goBack();

// Go forward
await page.goForward();

// Reload page
await page.reload();

// Wait for page load
await page.goto('https://example.com', { waitUntil: 'load' });
// Options: 'load', 'domcontentloaded', 'networkidle'
```

RAHUL SHETTY ACADEMY

Module 5: Assertions & Validations

Expect Assertions

Page Assertions

```
// Title
await expect(page).toHaveTitle('LoginPage Practise | Rahul Shetty Academy');
await expect(page).toHaveTitle(/LoginPage/);

// URL
await expect(page).toHaveURL('https://rahulshettyacademy.com/');
await expect(page).toHaveURL(/rahulshettyacademy/);
```

Element Assertions

```
// Visibility
await expect(page.locator('#username')).toBeVisible();
await expect(page.locator('#loading')).toBeHidden();

// Enabled/Disabled
await expect(page.locator('#submit')).toBeEnabled();
await expect(page.locator('#submit')).toBeDisabled();

// Text content
await expect(page.locator('.error')).toHaveText('Invalid credentials');
await expect(page.locator('.error')).toContainText('Invalid');

// Attribute
await expect(page.locator('#username')).toHaveAttribute('type', 'text');

// Count
await expect(page.locator('.product')).toHaveCount(5);

// Value (for inputs)
await expect(page.locator('#username')).toHaveValue('rahulshetty');
```

Soft Assertions (Continue on Failure)

```
await expect.soft(page.locator('#error')).toBeVisible();
await expect.soft(page.locator('.message')).toHaveText('Success');
// Test continues even if assertions fail
```

RAHUL SHETTY ACADEMY

Module 6: Waits & Synchronization

Auto-Wait (Built-in)

Playwright automatically waits for:

- Element to be visible
- Element to be enabled
- Element to be stable (not animating)

```
// Playwright waits automatically before click
await page.locator('#submit').click();
```

Explicit Waits

Wait for Element

```
// Wait for element to be visible
await page.locator('#successMessage').waitFor({ state: 'visible' });

// Wait for element to be hidden
await page.locator('#loader').waitFor({ state: 'hidden' });

// Wait for element to be attached
await page.locator('#dynamic').waitFor({ state: 'attached' });
```

Wait for Navigation

```
await page.waitForURL('**/dashboard');
await page.waitForLoadState('networkidle');
```

Wait for Timeout

```
await page.waitForTimeout(3000); // Not recommended, use sparingly
```

Wait for Function

```
await page.waitForFunction(() => document.querySelector('#status').innerText === 'Ready');
```

RAHUL SHETTY ACADEMY

Module 7: Handling Web Elements

Working with Multiple Elements

```
// Get all matching elements
const products = await page.locator('.product-card').all();
console.log(`Found ${products.length} products`);

// Count elements
const count = await page.locator('.product-card').count();

// Iterate through elements
for (const product of products) {
    console.log(await product.textContent());
}

// Find specific element
const allProducts = await page.locator('.product-card').allTextContents();
const targetProduct = page.locator('.product-card').filter({ hasText: 'iPhone' });
```

Handling Dynamic Elements

```
// Wait for element to appear
await page.locator('#dynamicElement').waitFor();

// Get text content
const text = await page.locator('#message').textContent();
const innerText = await page.locator('#message').innerText();

// Get attribute
const href = await page.locator('a').getAttribute('href');

// Check if element exists
const isVisible = await page.locator('#element').isVisible();
const isEnabled = await page.locator('#button').isEnabled();
```

Module 8: Frames & iFrames

RAHUL SHETTY ACADEMY

Switching to Frames

```
// Get frame by name
const frame = page.frame('frameName');
await frame.locator('#element').click();

// Get frame by URL
const frame2 = page.frame({ url: /iframe-url/ });

// Frame locator (Recommended)
await page.frameLocator('#iframeId').locator('#element').click();

// Nested frames
await page
    .frameLocator('#outerFrame')
    .frameLocator('#innerFrame')
    .locator('#element')
    .click();
```

Example

```
test('Handle iframe', async ({ page }) => {
    await page.goto('https://rahulshettyacademy.com/AutomationPractice/');

    // Work with iframe
    const framesPage = page.frameLocator('#courses-iframe');
    await framesPage.locator("a[href='lifetime-access']").first().click();
    const text = await framesPage.locator('.text h2').textContent();
    console.log(text);
});
```

Module 9: Alerts, Popups & Dialogs

Handling Alerts

```
// Accept alert
page.on('dialog', async dialog => {
    console.log(dialog.message());
    await dialog.accept();
});
```

RAHUL SHETTY ACADEMY

```
await page.locator('#alertBtn').click();

// Dismiss alert
page.on('dialog', async dialog => {
    await dialog.dismiss();
});

// Alert with input
page.on('dialog', async dialog => {
    await dialog.accept('My input text');
});

// Get alert type
page.on('dialog', async dialog => {
    console.log(dialog.type()); // alert, confirm, prompt
    await dialog.accept();
});
```

Handling New Windows/Tabs

```
// Listen for new page
const [newPage] = await Promise.all([
    page.waitForEvent('popup'),
    page.locator('#openWindow').click()
]);

// Work with new page
await newPage.waitForLoadState();
console.log(await newPage.title());
await newPage.close();
```

Handling File Upload

```
// Single file
await page.locator('#fileInput').setInputFiles('path/to/file.pdf');

// Multiple files
await page.locator('#fileInput').setInputFiles([
    'file1.pdf',
    'file2.pdf'
]);
```

RAHUL SHETTY ACADEMY

```
// Remove files
await page.locator('#fileInput').setInputFiles([]);
```

Handling Downloads

```
const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.locator('#downloadBtn').click()
]);

// Save file
await download.saveAs('downloads/' + download.suggestedFilename());

// Get file path
const path = await download.path();
```

Module 10: Screenshots & Videos

Screenshots

```
// Full page screenshot
await page.screenshot({ path: 'screenshot.png', fullPage: true });

// Element screenshot
await page.locator('.header').screenshot({ path: 'header.png' });

// Screenshot on test failure (in config)
// playwright.config.js
use: {
    screenshot: 'only-on-failure',
    video: 'retain-on-failure'
}
```

Videos

Configure in playwright.config.js:

```
use: {
```

RAHUL SHETTY ACADEMY

```
    video: 'on', // 'on', 'off', 'retain-on-failure', 'on-first-retry'
    trace: 'on-first-retry' // Generate trace files
}
```

Trace Viewer

```
// playwright.config.js
use: {
    trace: 'on-first-retry'
}
```

View trace:

```
npx playwright show-trace trace.zip
```

Module 11: Network Interception

Monitor Network

```
// Listen to all requests
page.on('request', request => {
    console.log('>>', request.method(), request.url());
});

// Listen to all responses
page.on('response', response => {
    console.log('<<', response.status(), response.url());
});
```

Mock API Response

```
await page.route('**/api/products', async route => {
    const json = [
        { id: 1, name: 'Product 1', price: 100 },
        { id: 2, name: 'Product 2', price: 200 }
    ];
    await route.fulfill({ json });
});
```

RAHUL SHETTY ACADEMY

```
await page.goto('https://example.com');
```

Abort Requests (Block CSS, Images)

```
await page.route('**/*.{png,jpg,jpeg}', route => route.abort());
await page.route('**/*.css', route => route.abort());
```

Modify Request

```
await page.route('**/api/login', async route => {
   const request = route.request();
   const headers = {
      ...request.headers(),
      'Authorization': 'Bearer fake-token'
   };
   await route.continue({ headers });
});
```

Module 12: API Testing with Playwright

Making API Requests

```
const { request } = require('@playwright/test');

test('API Test - GET', async ({ request }) => {
   const response = await request.get('https://reqres.in/api/users/2');
   expect(response.status()).toBe(200);

   const json = await response.json();
   expect(json.data.email).toBe('janet.weaver@reqres.in');
   console.log(json);
});
```

POST Request

```
test('API Test - POST', async ({ request }) => {
   const response = await request.post('https://reqres.in/api/users', {
      data: {
         name: 'Rahul',
```

RAHUL SHETTY ACADEMY

```
            job: 'Instructor'
        }
    });

    expect(response.status()).toBe(201);
    const json = await response.json();
    console.log(json);
});
```

API Testing with Authorization

```
test('API with Auth', async ({ request }) => {
    const response = await request.get('https://api.example.com/users', {
        headers: {
            'Authorization': 'Bearer token123',
            'Content-Type': 'application/json'
        }
    });

    expect(response.ok()).toBeTruthy();
});
```

Combine UI + API

```
test('Login via API, continue with UI', async ({ page, request }) => {
    // Login via API
    const response = await request.post('https://example.com/api/login', {
        data: { username: 'test', password: 'test123' }
    });
    const { token } = await response.json();

    // Set auth in browser
    await page.goto('https://example.com');
    await page.evaluate(token => {
        localStorage.setItem('authToken', token);
    }, token);

    await page.goto('https://example.com/dashboard');
});
```

RAHUL SHETTY ACADEMY

Module 13: Page Object Model (POM)

Why POM?

- **Reusability** - Use same page objects across tests
- **Maintainability** - Change locators in one place
- **Readability** - Tests are cleaner and easier to understand

Basic POM Structure

LoginPage.js

```
class LoginPage {
    constructor(page) {
        this.page = page;
        this.usernameInput = page.locator('#username');
        this.passwordInput = page.locator('#password');
        this.loginButton = page.locator('#signInBtn');
        this.errorMessage = page.locator('.alert-danger');
    }

    async navigate() {
        await this.page.goto('https://rahulshettyacademy.com/loginpagePractise/');
    }

    async login(username, password) {
        await this.usernameInput.fill(username);
        await this.passwordInput.fill(password);
        await this.loginButton.click();
    }

    async getErrorMessage() {
        return await this.errorMessage.textContent();
    }

    async isErrorVisible() {
        return await this.errorMessage.isVisible();
    }
}

module.exports = LoginPage;
```

RAHUL SHETTY ACADEMY

Using POM in Test

```
const { test, expect } = require('@playwright/test');
const LoginPage = require('./pages/LoginPage');

test('Login Test with POM', async ({ page }) => {
    const loginPage = new LoginPage(page);

    await loginPage.navigate();
    await loginPage.login('rahulshettyacademy', 'learning');

    // Verify navigation to next page
    await expect(page).toHaveURL(/shop/);
});
```

Advanced POM with TypeScript

pages/LoginPage.ts

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
    readonly page: Page;
    readonly usernameInput: Locator;
    readonly passwordInput: Locator;
    readonly loginButton: Locator;

    constructor(page: Page) {
        this.page = page;
        this.usernameInput = page.locator('#username');
        this.passwordInput = page.locator('#password');
        this.loginButton = page.locator('#signInBtn');
    }

    async navigate(): Promise<void> {
        await this.page.goto('https://rahulshettyacademy.com/loginpagePractise/');
    }

    async login(username: string, password: string): Promise<void> {
        await this.usernameInput.fill(username);
        await this.passwordInput.fill(password);
        await this.loginButton.click();
```

RAHUL SHETTY ACADEMY

```
    }
}
```

Module 14: Test Configuration

playwright.config.js

```
const { defineConfig, devices } = require('@playwright/test');

module.exports = defineConfig({
    // Test directory
    testDir: './tests',

    // Timeout per test
    timeout: 30000,

    // Retries on failure
    retries: 1,

    // Number of parallel workers
    workers: 3,

    // Reporter
    reporter: [
        ['html'],
        ['list'],
        ['junit', { outputFile: 'results.xml' }]
    ],

    use: {
        // Base URL
        baseURL: 'https://rahulshettyacademy.com',

        // Browser options
        headless: true,
        screenshot: 'only-on-failure',
        video: 'retain-on-failure',
        trace: 'on-first-retry',

        // Viewport
```

RAHUL SHETTY ACADEMY

```
    viewport: { width: 1280, height: 720 },

    // Ignore HTTPS errors
    ignoreHTTPSErrors: true,

    // Default timeout for actions
    actionTimeout: 10000,
  },

  // Configure projects for multiple browsers
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
    {
      name: 'mobile-chrome',
      use: { ...devices['Pixel 5'] },
    },
  ],
});
```

Module 15: Hooks & Fixtures

Test Hooks

```
const { test, expect } = require('@playwright/test');

test.describe('Login Tests', () => {
  // Before All - Run once before all tests
  test.beforeAll(async () => {
    console.log('Setup test suite');
```

RAHUL SHETTY ACADEMY

```
    });

    // Before Each - Run before each test
    test.beforeEach(async ({ page }) => {
        await page.goto('https://rahulshettyacademy.com/loginpagePractise/');
    });

    // After Each - Run after each test
    test.afterEach(async ({ page }) => {
        console.log('Test completed');
    });

    // After All - Run once after all tests
    test.afterAll(async () => {
        console.log('Cleanup test suite');
    });

    test('Test 1', async ({ page }) => {
        // Test code
    });

    test('Test 2', async ({ page }) => {
        // Test code
    });
});
```

Custom Fixtures

```
// fixtures.js
const base = require('@playwright/test');
const LoginPage = require('./pages/LoginPage');

exports.test = base.test.extend({
    loginPage: async ({ page }, use) => {
        const loginPage = new LoginPage(page);
        await loginPage.navigate();
        await use(loginPage);
    }
});

exports.expect = base.expect;
```

RAHUL SHETTY ACADEMY

```
// Use in test
const { test, expect } = require('./fixtures');

test('Login test with fixture', async ({ loginPage }) => {
    await loginPage.login('user', 'pass');
});
```

Module 16: Data-Driven Testing

Using JSON File

testData.json

```
{
    "users": [
        { "username": "user1", "password": "pass1" },
        { "username": "user2", "password": "pass2" },
        { "username": "user3", "password": "pass3" }
    ]
}
```

Test File

```
const { test } = require('@playwright/test');
const testData = require('./testData.json');

for (const user of testData.users) {
    test(`Login test for ${user.username}`, async ({ page }) => {
        await page.goto('https://example.com/login');
        await page.locator('#username').fill(user.username);
        await page.locator('#password').fill(user.password);
        await page.locator('#login').click();
    });
}
```

Using CSV with PapaParse

```
const fs = require('fs');
```

RAHUL SHETTY ACADEMY

```
const Papa = require('papaparse');

const csvData = fs.readFileSync('testdata.csv', 'utf-8');
const parsedData = Papa.parse(csvData, { header: true }).data;

for (const row of parsedData) {
    test(`Test with ${row.username}`, async ({ page }) => {
        // Use row.username, row.password, etc.
    });
}
```

Module 17: Parallel Execution

Configure Workers

```
// playwright.config.js
module.exports = defineConfig({
    workers: 5, // Run 5 tests in parallel
});
```

Parallel Tests in Same File

```
test.describe.configure({ mode: 'parallel' });

test.describe('Parallel Tests', () => {
    test('Test 1', async ({ page }) => { });
    test('Test 2', async ({ page }) => { });
    test('Test 3', async ({ page }) => { });
    // All run in parallel
});
```

Serial Tests (One After Another)

```
test.describe.configure({ mode: 'serial' });

test.describe('Serial Tests', () => {
    test('Test 1', async ({ page }) => { });
    test('Test 2', async ({ page }) => { });
    // Run one after another
```

RAHUL SHETTY ACADEMY

});

Module 18: Reporting

Built-in Reporters

```
// playwright.config.js
reporter: [
    ['list'], // Console output
    ['html', { open: 'never' }], // HTML report
    ['json', { outputFile: 'results.json' }], // JSON report
    ['junit', { outputFile: 'results.xml' }] // JUnit XML
]
```

View HTML Report

```
npx playwright show-report
```

Custom Reporter

```
class MyReporter {
    onBegin(config, suite) {
        console.log(`Starting tests: ${suite.allTests().length} tests`);
    }

    onTestEnd(test, result) {
        console.log(`${test.title}: ${result.status}`);
    }

    onEnd(result) {
        console.log(`Tests finished: ${result.status}`);
    }
}

module.exports = MyReporter;
```

RAHUL SHETTY ACADEMY

Module 19: Debugging Tests

Debug Mode

npx playwright test --debug

UI Mode (Recommended)

npx playwright test --ui

Features:

- Watch tests run
- Step through test
- Inspect locators
- View console logs

Playwright Inspector

await page.pause(); // Stops execution and opens inspector

Console Logs

page.on('console', msg => console.log(msg.text()));

Visual Debugging

// Slow down execution
await page.locator('#button').click({ delay: 1000 });

// Highlight element
await page.locator('#element').highlight();

Module 20: JavaScript Essentials for Playwright

Async/await

// All Playwright methods are async
async function runTest() {

RAHUL SHETTY ACADEMY

```
    await page.goto('url');
    await page.click('button');
}
```

Promises

```
// Wait for multiple actions
await Promise.all([
    page.waitForNavigation(),
    page.click('#submit')
]);
```

Array Methods

```
const products = await page.locator('.product').all();

// Map
const prices = await Promise.all(
    products.map(p => p.locator('.price').textContent())
);

// Filter
const expensiveProducts = prices.filter(price => parseFloat(price) > 100);

// Find
const product = products.find(async p => {
    const text = await p.textContent();
    return text.includes('iPhone');
});
```

Template Literals

```
const username = 'test@test.com';
await page.goto(`https://example.com/user/${username}`);
```

Destructuring

```
const { chromium } = require('@playwright/test');
const { username, password } = testData;
```

RAHUL SHETTY ACADEMY

Arrow Functions

```
test('My test', async ({ page }) => {
   await page.goto('url');
});
```

Module 21: Quick Reference Commands

```
# Install Playwright
npm init playwright@latest

# Run all tests
npx playwright test

# Run specific file
npx playwright test login.spec.js

# Run in headed mode
npx playwright test --headed

# Run in debug mode
npx playwright test --debug

# Run in UI mode
npx playwright test --ui

# Run specific browser
npx playwright test --project=chromium

# Generate code
npx playwright codegen https://example.com

# Show report
npx playwright show-report

# Show trace
npx playwright show-trace trace.zip

# Install browsers
npx playwright install
```

RAHUL SHETTY ACADEMY

Update Playwright
npm install -D @playwright/test@latest
npx playwright install

Module 22: INTERVIEW QUESTIONS & ANSWERS

PLAYWRIGHT BASICS

Q1: What is Playwright and why use it over Selenium?

A: Playwright is a modern automation framework by Microsoft for testing web applications.

Advantages over Selenium:

- **Auto-wait mechanism** - No explicit waits needed
- **Faster execution** - Better performance
- **Multi-browser support** - Chrome, Firefox, Safari (WebKit) from single codebase
- **API testing** - Built-in API testing capabilities
- **Network interception** - Mock/modify network requests
- **Browser contexts** - Isolated test environments
- **Better debugging** - Trace viewer, UI mode, inspector
- **Mobile emulation** - Easy device emulation
- **Parallel execution** - Built-in parallel test support
- **Better error messages** - More descriptive failure messages

Q2: What are Browser Contexts in Playwright?

A: Browser contexts are isolated browser sessions within the same browser instance. Each context has its own cookies, cache, and local storage.

Benefits:

- Run parallel tests without interference
- Save resources (don't need multiple browser instances)
- Test scenarios requiring multiple users simultaneously

Example:

```
const browser = await chromium.launch();
```

RAHUL SHETTY ACADEMY

```
// Create two isolated contexts
const context1 = await browser.newContext();
const context2 = await browser.newContext();

const page1 = await context1.newPage();
const page2 = await context2.newPage();

// Both can login as different users simultaneously
await page1.goto('https://example.com');
await page2.goto('https://example.com');
```

Q3: Explain Auto-wait in Playwright.

A: Playwright automatically waits for elements to be actionable before performing actions. It waits for:

- Element to be attached to DOM
- Element to be visible
- Element to be stable (not animating)
- Element to receive events (not obscured)
- Element to be enabled

Example:

```
// Playwright automatically waits before clicking
await page.locator('#submit').click(); // No explicit wait needed!

// Unlike Selenium where you'd need:
// WebDriverWait wait = new WebDriverWait(driver, 10);
// wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));
```

Q4: What are the different types of locators in Playwright?

A: Playwright provides multiple locator strategies:

1. **Role-based (Recommended)**

```
await page.getByRole('button', { name: 'Submit' });
await page.getByRole('textbox', { name: 'Email' });
```

RAHUL SHETTY ACADEMY

2. **Test ID (Best for automation)**

```
await page.getByTestId('login-button');
```

3. **Text-based**

```
await page.getByText('Sign In');
await page.getByLabel('Username');
await page.getByPlaceholder('Enter email');
```

4. **CSS Selectors**

```
await page.locator('#username');
await page.locator('.btn-primary');
```

5. **XPath (last resort)**

```
await page.locator('//button[@id="submit"]');
```

Priority: getByRole > getByTestId > getByLabel > getByPlaceholder > CSS > XPath

Q5: How do you handle multiple elements in Playwright?

A:

```
// Get all elements
const products = await page.locator('.product').all();

// Count elements
const count = await page.locator('.product').count();

// Get all text contents
const texts = await page.locator('.product').allTextContents();

// Iterate and interact
for (const product of products) {
```

RAHUL SHETTY ACADEMY

```
    const name = await product.textContent();
    if (name.includes('iPhone')) {
        await product.click();
        break;
    }
}

// Filter elements
const iphone = page.locator('.product').filter({ hasText: 'iPhone' });
await iphone.click();

// Nth element
await page.locator('.product').nth(2).click(); // 3rd element
await page.locator('.product').first().click();
await page.locator('.product').last().click();
```

Q6: Explain different wait strategies in Playwright.

A:

1. Auto-wait (Default):

```
await page.locator('#button').click(); // Automatically waits
```

2. Wait for element state:

```
await page.locator('#element').waitFor({ state: 'visible' });
await page.locator('#loader').waitFor({ state: 'hidden' });
await page.locator('#element').waitFor({ state: 'attached' });
```

3. Wait for URL:

```
await page.waitForURL('**/dashboard');
await page.waitForURL(/admin/);
```

4. Wait for load state:

```
await page.waitForLoadState('load'); // Page load event
await page.waitForLoadState('domcontentloaded'); // DOM ready
```

RAHUL SHETTY ACADEMY

```
await page.waitForLoadState('networkidle'); // No network activity
```

5. Wait for function:

```
await page.waitForFunction(() => {
   return document.querySelector('#status').innerText === 'Ready';
});
```

6. Wait for timeout (not recommended):

```
await page.waitForTimeout(3000);
```

Q7: How do you handle iframes in Playwright?

A:

```
// Method 1: Frame locator (Recommended)
await page.frameLocator('#iframeId').locator('#element').click();

// Method 2: Get frame by name/URL
const frame = page.frame('frameName');
await frame.locator('#element').click();

// Method 3: Nested iframes
await page
   .frameLocator('#outerFrame')
   .frameLocator('#innerFrame')
   .locator('#button')
   .click();

// No need to switch back to main content!
await page.locator('#mainPageElement').click();
```

Q8: How do you handle alerts/dialogs in Playwright?

A:

```
// Listen for dialog before triggering
```

RAHUL SHETTY ACADEMY

```
page.on('dialog', async dialog => {
   console.log(dialog.message());
   console.log(dialog.type()); // alert, confirm, prompt

   await dialog.accept(); // Click OK
   // await dialog.dismiss(); // Click Cancel
   // await dialog.accept('input text'); // For prompts
});

await page.locator('#alertButton').click();

// One-time listener
page.once('dialog', async dialog => {
   await dialog.accept();
});
```

Q9: How do you handle multiple windows/tabs in Playwright?

A:

```
// Method 1: Using Promise.all
const [newPage] = await Promise.all([
   page.waitForEvent('popup'),
   page.locator('#openNewWindow').click()
]);

await newPage.waitForLoadState();
console.log(await newPage.title());
await newPage.locator('#element').click();
await newPage.close();

// Method 2: Using context
const pages = context.pages();
console.log(`Total pages: ${pages.length}`);

// Switch between pages
await pages[0].bringToFront();
await pages[1].bringToFront();
```

RAHUL SHETTY ACADEMY

Q10: How do you handle file uploads and downloads?

A:

File Upload:

```
// Single file
await page.locator('#fileInput').setInputFiles('path/to/file.pdf');

// Multiple files
await page.locator('#fileInput').setInputFiles([
   'file1.pdf',
   'file2.pdf'
]);

// Remove file selection
await page.locator('#fileInput').setInputFiles([]);

// Upload with file chooser
const [fileChooser] = await Promise.all([
   page.waitForEvent('filechooser'),
   page.locator('#uploadButton').click()
]);
await fileChooser.setFiles('path/to/file.pdf');
```

File Download:

```
const [download] = await Promise.all([
   page.waitForEvent('download'),
   page.locator('#downloadButton').click()
]);

// Save with suggested filename
await download.saveAs('downloads/' + download.suggestedFilename());

// Get file path
const path = await download.path();
console.log(path);

// Verify download
const fileName = download.suggestedFilename();
expect(fileName).toBe('report.pdf');
```

RAHUL SHETTY ACADEMY

PLAYWRIGHT ADVANCED

Q11: What is Page Object Model in Playwright?

A: POM is a design pattern where each page of the application has a corresponding page class containing locators and methods.

Benefits:

- Code reusability
- Easy maintenance
- Better readability
- Separation of test logic and page structure

Example:

```
// pages/LoginPage.js
class LoginPage {
    constructor(page) {
        this.page = page;
        this.usernameInput = page.locator('#username');
        this.passwordInput = page.locator('#password');
        this.loginButton = page.locator('#signInBtn');
    }

    async navigate() {
        await this.page.goto('/login');
    }

    async login(username, password) {
        await this.usernameInput.fill(username);
        await this.passwordInput.fill(password);
        await this.loginButton.click();
    }
}

// Test file
test('Login test', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.navigate();
```

RAHUL SHETTY ACADEMY

```
    await loginPage.login('user', 'pass');
});
```

Q12: How do you perform API testing in Playwright?

A:

```
test('GET API test', async ({ request }) => {
    const response = await request.get('https://api.example.com/users/1');

    expect(response.status()).toBe(200);
    expect(response.ok()).toBeTruthy();

    const json = await response.json();
    expect(json.id).toBe(1);
    expect(json.email).toContain('@');
});

test('POST API test', async ({ request }) => {
    const response = await request.post('https://api.example.com/users', {
        data: {
            name: 'John',
            email: 'john@test.com'
        },
        headers: {
            'Content-Type': 'application/json',
            'Authorization': 'Bearer token123'
        }
    });

    expect(response.status()).toBe(201);
    const body = await response.json();
    expect(body.name).toBe('John');
});

// Combine UI and API
test('Create user via API, verify in UI', async ({ page, request }) => {
    // Create via API
    const response = await request.post('/api/users', {
        data: { name: 'Test User' }
    });
```

RAHUL SHETTY ACADEMY

```
    const user = await response.json();

    // Verify in UI
    await page.goto('/users');
    await expect(page.locator(`text=${user.name}`)).toBeVisible();
});
```

Q13: How do you mock API responses in Playwright?

A:

```
test('Mock API response', async ({ page }) => {
    // Mock the API endpoint
    await page.route('**/api/products', async route => {
        const json = [
            { id: 1, name: 'Product 1', price: 100 },
            { id: 2, name: 'Product 2', price: 200 }
        ];
        await route.fulfill({
            status: 200,
            json
        });
    });

    await page.goto('https://example.com/products');

    // Page will use mocked data
    await expect(page.locator('.product')).toHaveCount(2);
});

// Mock with file
await page.route('**/api/users', async route => {
    await route.fulfill({
        path: 'mocks/users.json'
    });
});

// Modify existing response
await page.route('**/api/products', async route => {
    const response = await route.fetch();
    const json = await response.json();
```

RAHUL SHETTY ACADEMY

```
    // Modify response
    json.forEach(product => product.price = 0);

    await route.fulfill({ json });
});
```

Q14: How do you handle network requests/responses?

A:

```
// Monitor all requests
page.on('request', request => {
    console.log('>>', request.method(), request.url());
});

// Monitor all responses
page.on('response', response => {
    console.log('<<', response.status(), response.url());
});

// Block requests (images, CSS)
await page.route('**/*.{png,jpg,jpeg,gif}', route => route.abort());
await page.route('**/*.css', route => route.abort());

// Modify request headers
await page.route('**/api/**', async route => {
    const headers = {
        ...route.request().headers(),
        'Authorization': 'Bearer fake-token',
        'Custom-Header': 'value'
    };
    await route.continue({ headers });
});

// Wait for specific API call
const responsePromise = page.waitForResponse(
    response => response.url().includes('/api/users') && response.status() === 200
);
await page.locator('#loadUsers').click();
const response = await responsePromise;
```

RAHUL SHETTY ACADEMY



```
const users = await response.json();
```

Q15: What are fixtures in Playwright?

A: Fixtures are a way to set up test environment and share common objects across tests.

Built-in fixtures:

- `page` - Browser page instance
- `context` - Browser context
- `browser` - Browser instance
- `request` - API request context

Custom fixtures:

```
// fixtures.js
const base = require('@playwright/test');
const { LoginPage } = require('./pages/LoginPage');

exports.test = base.test.extend({
  // Auto-navigate to login page
  loginPage: async ({ page }, use) => {
    const loginPage = new LoginPage(page);
    await loginPage.navigate();
    await use(loginPage);
  },

  // Logged in user fixture
  authenticatedPage: async ({ page }, use) => {
    await page.goto('/login');
    await page.fill('#username', 'testuser');
    await page.fill('#password', 'testpass');
    await page.click('#login');
    await page.waitForURL('/dashboard');
    await use(page);
  }
});

exports.expect = base.expect;

// Use in test
```

RAHUL SHETTY ACADEMY

```
const { test, expect } = require('./fixtures');

test('Test with fixture', async ({ loginPage }) => {
  await loginPage.login('user', 'pass');
});

test('Test authenticated', async ({ authenticatedPage }) => {
  // Already logged in!
  await expect(authenticatedPage.locator('#username')).toBeVisible();
});
```

Q16: How do you implement data-driven testing?

A:

Method 1: JSON file

```
// testData.json
{
  "users": [
    { "username": "user1", "password": "pass1", "expected": "Dashboard" },
    { "username": "user2", "password": "pass2", "expected": "Dashboard" }
  ]
}

// test file
const testData = require('./testData.json');

for (const user of testData.users) {
  test(`Login test for ${user.username}`, async ({ page }) => {
    await page.goto('/login');
    await page.fill('#username', user.username);
    await page.fill('#password', user.password);
    await page.click('#login');
    await expect(page).toHaveTitle(user.expected);
  });
}
```

Method 2: Array in test

RAHUL SHETTY ACADEMY

```
const testCases = [
    { input: 'test1', expected: 'result1' },
    { input: 'test2', expected: 'result2' }
];

test.describe('Search tests', () => {
    for (const tc of testCases) {
        test(`Search for ${tc.input}`, async ({ page }) => {
            await page.goto('/search');
            await page.fill('#search', tc.input);
            await expect(page.locator('.result')).toHaveText(tc.expected);
        });
    }
});
```

Method 3: CSV with PapaParse

```
const fs = require('fs');
const Papa = require('papaparse');

const csv = fs.readFileSync('data.csv', 'utf-8');
const data = Papa.parse(csv, { header: true }).data;

for (const row of data) {
    test(`Test ${row.testName}`, async ({ page }) => {
        // Use row.username, row.password, etc.
    });
}
```

Q17: How do you configure parallel execution?

A:

playwright.config.js:

```
module.exports = defineConfig({
    // Number of parallel workers
    workers: 4, // or process.env.CI ? 2 : 4

    // Fully parallel mode
```

RAHUL SHETTY ACADEMY

```
    fullyParallel: true,

    // Retry failed tests
    retries: process.env.CI ? 2 : 0,
});
```

Control at test level:

```
// Parallel tests (default)
test.describe.configure({ mode: 'parallel' });
test.describe('Parallel tests', () => {
    test('Test 1', async ({ page }) => { });
    test('Test 2', async ({ page }) => { });
});

// Serial tests (one after another)
test.describe.configure({ mode: 'serial' });
test.describe('Serial tests', () => {
    test('Step 1', async ({ page }) => { });
    test('Step 2', async ({ page }) => { });
});

// Limit workers for specific file
test.describe.configure({ workers: 1 });
```

Run parallel tests:

```
# Use all workers
npx playwright test

# Specify workers
npx playwright test --workers=3

# Disable parallel (run one at a time)
npx playwright test --workers=1
```

Q18: What are different assertion methods in Playwright?

A:

RAHUL SHETTY ACADEMY

Page Assertions:

```
await expect(page).toHaveTitle('Expected Title');
await expect(page).toHaveTitle(/regex/);
await expect(page).toHaveURL('https://example.com');
await expect(page).toHaveURL(/pattern/);
```

Element Assertions:

```
// Visibility
await expect(locator).toBeVisible();
await expect(locator).toBeHidden();

// State
await expect(locator).toBeEnabled();
await expect(locator).toBeDisabled();
await expect(locator).toBeChecked();
await expect(locator).toBeEditable();
await expect(locator).toBeFocused();

// Text
await expect(locator).toHaveText('exact text');
await expect(locator).toHaveText(/regex/);
await expect(locator).toContainText('partial');

// Attributes
await expect(locator).toHaveAttribute('attr', 'value');
await expect(locator).toHaveClass('class-name');
await expect(locator).toHaveClass(/pattern/);
await expect(locator).toHaveId('element-id');

// Value
await expect(locator).toHaveValue('value');
await expect(locator).toHaveValues(['val1', 'val2']);

// Count
await expect(locator).toHaveCount(5);

// CSS
await expect(locator).toHaveCSS('color', 'rgb(0, 0, 0)');
```

RAHUL SHETTY ACADEMY

Soft Assertions (continue on failure):

```
await expect.soft(locator).toBeVisible();
await expect.soft(page).toHaveTitle('Title');
// Test continues even if assertions fail
```

Custom timeout:

```
await expect(locator).toBeVisible({ timeout: 10000 });
```

Q19: How do you handle screenshots and videos?

A:

Screenshots:

```
// Full page screenshot
await page.screenshot({ path: 'screenshot.png', fullPage: true });

// Element screenshot
await page.locator('.header').screenshot({ path: 'header.png' });

// Screenshot to buffer
const buffer = await page.screenshot();

// Screenshot on failure (config)
// playwright.config.js
use: {
   screenshot: 'only-on-failure', // or 'on', 'off'
}

// In test
test('My test', async ({ page }, testInfo) => {
   // Take screenshot programmatically
   await testInfo.attach('screenshot', {
      body: await page.screenshot(),
      contentType: 'image/png'
   });
});
```

RAHUL SHETTY ACADEMY

Videos:

```
// playwright.config.js
use: {
   video: 'on', // 'on', 'off', 'retain-on-failure', 'on-first-retry'
}

// Access video path in test
test('My test', async ({ page }, testInfo) => {
   // Test code

   // After test, video path available
   console.log(await page.video().path());
});
```

Q20: What is Trace Viewer and how to use it?

A: Trace Viewer is a debugging tool that records test execution with snapshots, network activity, console logs, and actions.

Enable tracing:

```
// playwright.config.js
use: {
   trace: 'on-first-retry', // 'on', 'off', 'retain-on-failure'
}
```

View trace:

```
npx playwright show-trace trace.zip
```

Features:

- Timeline of all actions
- DOM snapshots at each step
- Network activity
- Console logs
- Source code
- Screenshots
- Before/After states

RAHUL SHETTY ACADEMY

Programmatic tracing:

```
test('My test', async ({ page, context }) => {
   await context.tracing.start({ screenshots: true, snapshots: true });

   // Test steps
   await page.goto('https://example.com');

   await context.tracing.stop({ path: 'trace.zip' });
});
```

Q21: How do you handle authentication/login state?

A:

Method 1: Save auth state

```
// auth.setup.js
const { test as setup } = require('@playwright/test');

setup('authenticate', async ({ page }) => {
   await page.goto('/login');
   await page.fill('#username', 'user');
   await page.fill('#password', 'pass');
   await page.click('#login');

   // Save signed-in state
   await page.context().storageState({ path: 'auth.json' });
});

// playwright.config.js
module.exports = defineConfig({
   projects: [
      { name: 'setup', testMatch: /auth\.setup\.js/ },
      {
         name: 'chromium',
         use: {
            ...devices['Desktop Chrome'],
            storageState: 'auth.json' // Reuse auth state
         },
         dependencies: ['setup']
      }
```

RAHUL SHETTY ACADEMY

```
  ]
});

// Now all tests start logged in!
test('Dashboard test', async ({ page }) => {
  await page.goto('/dashboard');
  // Already authenticated!
});
```

Method 2: Login via API

```
test.use({
  storageState: async ({}, use) => {
    const response = await apiRequest.post('/api/login', {
      data: { username: 'user', password: 'pass' }
    });
    const { token } = await response.json();

    await use({
      cookies: [],
      origins: [{
        origin: 'https://example.com',
        localStorage: [{
          name: 'token',
          value: token
        }]
      }]
    });
  }
});
```

Method 3: Fixture

```
const test = base.test.extend({
  authenticatedPage: async ({ page }, use) => {
    await page.goto('/login');
    await page.fill('#username', 'user');
    await page.fill('#password', 'pass');
    await page.click('#login');
    await use(page);
  }
```

RAHUL SHETTY ACADEMY

});

Q22: How do you debug Playwright tests?

A:

1. UI Mode (Best for debugging):

npx playwright test --ui

Features: Watch tests, step through, inspect locators, time travel

2. Debug Mode:

npx playwright test --debug

3. Playwright Inspector:

await page.pause(); // Stops here and opens inspector

4. Headed Mode:

npx playwright test --headed --slowmo=1000

5. Console Logs:

page.on('console', msg => console.log('PAGE LOG:', msg.text()));
console.log(await page.title());

6. Screenshots:

await page.screenshot({ path: 'debug.png' });

7. Trace Viewer:

npx playwright show-trace trace.zip

RAHUL SHETTY ACADEMY

8. VSCode Debugger:

```
// .vscode/launch.json
{
   "type": "node",
   "request": "launch",
   "name": "Debug Playwright",
   "program": "${workspaceFolder}/node_modules/@playwright/test/cli.js",
   "args": ["test", "--headed"]
}
```

Q23: What are test hooks in Playwright?

A:

```
test.describe('Test Suite', () => {
   // Runs ONCE before all tests in this describe block
   test.beforeAll(async ({ browser }) => {
      console.log('Setup database');
   });

   // Runs before EACH test
   test.beforeEach(async ({ page }) => {
      await page.goto('/');
   });

   // Runs after EACH test
   test.afterEach(async ({ page }, testInfo) => {
      if (testInfo.status !== 'passed') {
         await page.screenshot({ path: `failure-${testInfo.title}.png` });
      }
   });

   // Runs ONCE after all tests
   test.afterAll(async () => {
      console.log('Cleanup database');
   });

   test('Test 1', async ({ page }) => { });
   test('Test 2', async ({ page }) => { });
});
```

RAHUL SHETTY ACADEMY

Execution order:

```
beforeAll
  beforeEach
    test 1
  afterEach
  beforeEach
    test 2
  afterEach
afterAll
```

Q24: How do you handle dynamic tables/webtables?

A:

```
// Get all rows
const rows = await page.locator('table tbody tr').all();
console.log(`Total rows: ${rows.length}`);

// Get specific cell data
const cellData = await page.locator('table tbody tr:nth-child(2) td:nth-child(3)').textContent();

// Iterate through rows
for (const row of rows) {
    const cells = await row.locator('td').all();
    for (const cell of cells) {
        console.log(await cell.textContent());
    }
}

// Find row containing specific text
const targetRow = page.locator('table tbody tr').filter({ hasText: 'John' });
await targetRow.locator('.edit-button').click();

// Get all column data
const names = await page.locator('table tbody tr td:nth-child(1)').allTextContents();

// Click button in specific row
await page.locator('table tbody tr')
    .filter({ hasText: 'test@example.com' })
    .locator('.delete-button')
    .click();
```

RAHUL SHETTY ACADEMY

```
// Sort and verify
await page.locator('th.name').click(); // Click header to sort
const sortedNames = await page.locator('tbody tr td:nth-child(1)').allTextContents();
expect(sortedNames).toEqual([...sortedNames].sort());
```

Q25: What are the different reporters in Playwright?

A:

Built-in Reporters:

```
// playwright.config.js
reporter: [
  ['list'], // Console output during test run
  ['html'], // HTML report (default)
  ['json', { outputFile: 'results.json' }],
  ['junit', { outputFile: 'results.xml' }],
  ['dot'], // Minimal console output
  ['line'], // One line per test
  ['github'], // GitHub Actions annotations
]
```

Custom Reporter:

```
class CustomReporter {
  onBegin(config, suite) {
    console.log(`Starting ${suite.allTests().length} tests`);
  }

  onTestBegin(test) {
    console.log(`Starting ${test.title}`);
  }

  onTestEnd(test, result) {
    console.log(`Finished ${test.title}: ${result.status}`);
  }

  onEnd(result) {
    console.log(`Status: ${result.status}`);
  }
```

RAHUL SHETTY ACADEMY

}

module.exports = CustomReporter;

// Use in config
reporter: [['./customReporter.js']]

View HTML report:

npx playwright show-report

REAL-WORLD SCENARIOS

Q26: How would you test file download functionality?

A:

```
test('Download file and verify', async ({ page }) => {
    const [download] = await Promise.all([
        page.waitForEvent('download'),
        page.locator('#downloadButton').click()
    ]);

    // Verify filename
    expect(download.suggestedFilename()).toBe('report.pdf');

    // Save file
    const path = await download.path();
    await download.saveAs('./downloads/' + download.suggestedFilename());

    // Verify file exists and has content
    const fs = require('fs');
    const stats = fs.statSync(path);
    expect(stats.size).toBeGreaterThan(0);
});
```

Q27: How do you test responsive/mobile designs?

A:

RAHUL SHETTY ACADEMY

```
// Method 1: Use device descriptor
const { devices } = require('@playwright/test');

test.use({
  ...devices['iPhone 12']
});

test('Mobile test', async ({ page }) => {
  await page.goto('/');
  // Page is now in iPhone 12 viewport
});

// Method 2: Custom viewport
test.use({
  viewport: { width: 375, height: 667 }
});

// Method 3: Change viewport in test
await page.setViewportSize({ width: 1920, height: 1080 });

// playwright.config.js - Multiple devices
projects: [
  {
    name: 'Mobile Chrome',
    use: { ...devices['Pixel 5'] }
  },
  {
    name: 'Mobile Safari',
    use: { ...devices['iPhone 12'] }
  },
  {
    name: 'Desktop',
    use: { viewport: { width: 1920, height: 1080 } }
  }
]
```

Q28: How would you implement end-to-end e-commerce flow?

A:

```
test('E2E: Browse, Add to Cart, Checkout', async ({ page }) => {
  // Home page
```

RAHUL SHETTY ACADEMY

```
    await page.goto('/');
    await expect(page).toHaveTitle(/Shop/);

    // Search product
    await page.fill('#search', 'iPhone');
    await page.press('#search', 'Enter');

    // Select product
    const products = await page.locator('.product-card').all();
    expect(products.length).toBeGreaterThan(0);
    await products[0].locator('.add-to-cart').click();

    // Verify cart count
    await expect(page.locator('.cart-count')).toHaveText('1');

    // Go to cart
    await page.locator('.cart-icon').click();
    await expect(page).toHaveURL(/cart/);

    // Proceed to checkout
    await page.locator('#checkout-button').click();

    // Fill shipping details
    await page.fill('#firstName', 'John');
    await page.fill('#lastName', 'Doe');
    await page.fill('#email', 'john@test.com');
    await page.fill('#address', '123 Main St');

    // Submit order
    await page.locator('#place-order').click();

    // Verify confirmation
    await expect(page.locator('.success-message')).toBeVisible();
    await expect(page.locator('.order-number')).toBeVisible();
});
```

Q29: How do you handle race conditions?

A:

```
// Problem: Element might not be ready
// ❌ Bad
```

RAHUL SHETTY ACADEMY

```
await page.click('#button');
await page.fill('#input', 'text'); // Might fail if input appears late

// ✅ Good - Wait for element
await page.locator('#button').click();
await page.locator('#input').waitFor({ state: 'visible' });
await page.locator('#input').fill('text');

// ✅ Better - Auto-wait handles it
await page.locator('#button').click();
await page.locator('#input').fill('text'); // Automatically waits!

// For network race conditions
await Promise.all([
    page.waitForResponse(resp => resp.url().includes('/api/data')),
    page.locator('#loadData').click()
]);

// For multiple async operations
const [response1, response2] = await Promise.all([
    page.waitForResponse('/api/users'),
    page.waitForResponse('/api/products'),
    page.goto('/dashboard')
]);
```

Q30: How would you implement CI/CD with Playwright?

A:

GitHub Actions (.github/workflows/playwright.yml):

```
name: Playwright Tests

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-latest
```

RAHUL SHETTY ACADEMY

```
    steps:
    - uses: actions/checkout@v3

    - uses: actions/setup-node@v3
      with:
        node-version: 18

    - name: Install dependencies
      run: npm ci

    - name: Install Playwright Browsers
      run: npx playwright install --with-deps

    - name: Run Playwright tests
      run: npx playwright test

    - uses: actions/upload-artifact@v3
      if: always()
      with:
        name: playwright-report
        path: playwright-report/
        retention-days: 30
```

Jenkins Pipeline:

```
pipeline {
    agent any

    stages {
        stage('Install') {
            steps {
                sh 'npm ci'
                sh 'npx playwright install --with-deps'
            }
        }

        stage('Test') {
            steps {
                sh 'npx playwright test'
            }
```

RAHUL SHETTY ACADEMY

```
        }
    }

    post {
        always {
            publishHTML([
                reportDir: 'playwright-report',
                reportFiles: 'index.html',
                reportName: 'Playwright Report'
            ])
        }
    }
}
```

playwright.config.js for CI:

```
module.exports = defineConfig({
    workers: process.env.CI ? 2 : undefined,
    retries: process.env.CI ? 2 : 0,
    reporter: process.env.CI
        ? [['html'], ['github']]
        : [['html']],
});
```

Module 23: RESUME GUIDE FOR PLAYWRIGHT TESTERS

Professional Summary Examples

For Junior Level Experience:

Detail-oriented QA Automation Engineer with 1+ year of experience in web automation testing using Playwright and JavaScript/TypeScript. Proficient in designing and implementing automated test frameworks using Page Object Model. Experienced in API testing, cross-browser testing, and CI/CD integration. Strong understanding of modern testing practices and software development lifecycle.

For Mid-Senior Level Experience:

RAHUL SHETTY ACADEMY

Results-driven Senior QA Automation Engineer with 4+ years of experience specializing in Playwright automation testing. Expert in building scalable test automation frameworks from scratch using JavaScript/TypeScript and POM design pattern. Proven track record of reducing manual testing efforts by 70% and improving test coverage by 85%. Experienced in leading automation initiatives, mentoring junior testers, and implementing CI/CD pipelines with GitHub Actions and Jenkins.

Technical Skills Section

Automation Testing:

- Playwright (JavaScript/TypeScript)
- Selenium WebDriver
- API Testing with Playwright
- Cross-browser Testing (Chrome, Firefox, Safari)
- Mobile Testing & Responsive Design Testing

Frameworks & Patterns:

- Page Object Model (POM)
- Data-Driven Testing
- Keyword-Driven Framework
- Behavior-Driven Development (BDD)
- Hybrid Framework Design

Programming Languages:

- JavaScript (Advanced)
- TypeScript (Intermediate)
- Node.js
- HTML/CSS (for locator strategies)

Tools & Technologies:

- Git/GitHub
- VS Code
- Playwright Inspector & Trace Viewer
- Network Interception & Mocking
- Screenshot & Video Recording

RAHUL SHETTY ACADEMY

CI/CD & DevOps:

- GitHub Actions
- Jenkins
- Docker
- Azure DevOps

Testing Types:

- Functional Testing
- Regression Testing
- Smoke Testing
- API Testing
- End-to-End Testing
- Visual Regression Testing

Reporting & Documentation:

- HTML Reports
- Allure Reports
- Test Case Documentation
- Bug Reporting (Jira, Azure DevOps)

Professional Experience - Project Examples

Example 1: E-Commerce Automation

QA Automation Engineer | XYZ Company | Jan 2023 - Present

• Designed and implemented comprehensive Playwright automation framework for e-commerce application covering 200+ test scenarios
• Reduced regression testing time from 40 hours to 4 hours (90% reduction)
• Implemented Page Object Model architecture improving code reusability by 60%
• Integrated API testing with UI testing for end-to-end validation
• Set up parallel execution reducing overall test execution time by 75%
• Configured CI/CD pipeline with GitHub Actions for automated test execution
• Implemented network mocking for testing edge cases and failure scenarios
• Achieved 85% test coverage across critical user journeys
• Mentored 3 junior automation engineers on Playwright best practices

Technologies: Playwright, JavaScript, Node.js, GitHub Actions, POM

RAHUL SHETTY ACADEMY

Example 2: Banking Application

Senior QA Automation Engineer | ABC Bank | Jun 2021 - Dec 2022

• Led automation testing initiative for online banking platform using Playwright
• Developed data-driven testing framework with JSON/CSV data sources
• Implemented authentication state management for faster test execution
• Created custom fixtures and utilities for common banking operations
• Integrated Playwright with existing Jenkins pipeline
• Implemented screenshot and video capture for failed test scenarios
• Performed API testing for 50+ banking APIs
• Conducted cross-browser testing across Chrome, Firefox, and Safari
• Reduced production defects by 40% through comprehensive test coverage

Technologies: Playwright, TypeScript, Jenkins, Docker, Allure Reports

Example 3: SaaS Product Testing

QA Automation Engineer | Tech Startup | Mar 2022 - Present

• Built Playwright automation suite from scratch for SaaS application
• Implemented responsive design testing for mobile and tablet viewports
• Created network interception strategies for testing API failure scenarios
• Designed modular test architecture using TypeScript and POM pattern
• Integrated Playwright tests with Azure DevOps CI/CD pipeline
• Implemented visual regression testing for UI components
• Achieved 70% automation coverage within 6 months
• Reduced manual testing effort by 60%

Technologies: Playwright, TypeScript, Azure DevOps, Node.js

Key Achievements to Highlight

Quantifiable Metrics:

- "Reduced test execution time by X%"
- "Improved test coverage from X% to Y%"
- "Automated X number of test cases"

RAHUL SHETTY ACADEMY

- "Reduced manual testing effort by X hours per sprint"
- "Decreased production bugs by X%"
- "Mentored X team members"

Technical Achievements:

- "Designed and implemented scalable automation framework from scratch"
- "Integrated Playwright with CI/CD pipeline (GitHub Actions/Jenkins)"
- "Implemented parallel execution reducing execution time by X%"
- "Built reusable component library reducing script development time by X%"
- "Established API testing strategy covering X endpoints"

Process Improvements:

- "Introduced Playwright to replace legacy Selenium framework"
- "Established automation testing best practices and coding standards"
- "Created comprehensive test documentation and runbooks"
- "Implemented test reporting dashboard for stakeholders"

Projects Section Format

Project Name: E-Commerce Web Application Automation
Duration: Jan 2023 - Jun 2023
Role: Lead QA Automation Engineer

Description:
Automated testing for full-stack e-commerce platform with complex user workflows including product search, cart management, checkout, and payment processing.

Responsibilities:
• Framework design and implementation using Playwright and TypeScript
• Page Object Model architecture with 50+ page classes
• API testing integration for backend services
• Cross-browser testing across Chrome, Firefox, Safari
• Mobile responsive testing for iOS and Android viewports
• CI/CD integration with GitHub Actions
• Test data management using JSON files

Achievements:
• Automated 200+ test scenarios covering critical user journeys
• Reduced regression cycle time from 2 days to 4 hours
• Achieved 85% test coverage

RAHUL SHETTY ACADEMY

• Identified and reported 45+ defects during automation development

Technologies: Playwright, TypeScript, JavaScript, Node.js, GitHub Actions,
Page Object Model, API Testing

Certifications (if available)

- Microsoft Certified: Azure Fundamentals
- ISTQB Certified Tester - Foundation Level
- Playwright Certification (if any)
- JavaScript/TypeScript Certifications
- Agile/Scrum Certifications

Sample Interview Answers for Behavioral Questions

Q: "Tell me about a challenging automation scenario you faced and how you solved it."

A: "In my recent project, we had to automate a complex multi-step checkout flow with dynamic payment methods that loaded asynchronously. The challenge was handling race conditions where payment options would appear at different times based on user location and cart value.

I solved this by:

1. Implementing Playwright's auto-wait mechanism effectively
2. Using network interception to mock payment gateway responses for consistent testing
3. Creating custom wait utilities for specific business logic
4. Implementing parallel test execution with different scenarios
5. Using trace viewer to debug timing issues

The result was a stable test suite with 95% pass rate that caught several edge cases in production before release."

Q: "Why did you choose Playwright over other automation tools?"

A: "I chose Playwright because:

1. **Auto-wait mechanism** - Eliminates flaky tests caused by timing issues

RAHUL SHETTY ACADEMY

2. **Fast execution** - Significantly faster than Selenium
3. **Built-in API testing** - Can test both UI and API in same framework
4. **Better debugging** - Trace viewer and UI mode make debugging much easier
5. **Network interception** - Essential for testing various API scenarios
6. **Modern architecture** - Browser contexts allow true test isolation
7. **Multi-browser support** - Test across browsers from single codebase
8. **Active development** - Regular updates from Microsoft

In my current project, Playwright reduced our test execution time by 60% compared to our previous Selenium suite."

Q: "How do you ensure your automation tests are maintainable?"

A: "I follow these practices:

1. **Page Object Model** - Separate page logic from test logic
2. **Meaningful naming** - Clear, descriptive names for methods and variables
3. **Reusable components** - Common utilities for repeated actions
4. **Data-driven approach** - Externalize test data from test scripts
5. **Custom fixtures** - Set up common test preconditions
6. **Clear folder structure** - Organized pages, tests, utilities, data
7. **Code reviews** - Regular peer reviews of automation code
8. **Documentation** - Comment complex logic and maintain README
9. **Small, focused tests** - Each test validates one specific functionality
10. **Version control** - Proper Git workflow with meaningful commits

This approach has helped my team maintain a test suite of 500+ tests with minimal maintenance overhead."

Resume Do's :

- ✅ Use action verbs (Designed, Implemented, Automated, Reduced, Improved)
- ✅ Quantify achievements with numbers and percentages
- ✅ Highlight Playwright-specific features (auto-wait, network mocking, trace viewer)
- ✅ Mention both UI and API testing experience
- ✅ Include CI/CD integration experience
- ✅ List relevant projects with technical details
- ✅ Keep it concise (1-2 pages)
- ✅ Tailor resume for each job application

RAHUL SHETTY ACADEMY

RAHUL SHETTY ACADEMY